Assignment 5: Optimising Attention on a CPU

CS6886: Systems for Deep Learning — Name: Chirag Roll no.: DA25M008 Repo: https://github.com/chirag3211/Attention-Optimization-on-CPU

> All measurements: native Windows 11 (no WSL), MSYS2 clang 23 (`-O3 -march=native` , OpenMP via LLVM libomp), plugged in, “Best performance” power mode, background applications closed. Every point is the median of 5 invocations of the given driver, which itself reports the median of 5 timed runs after one warm-up. GFLOP/s is the driver’s count, $4N^2d$ FLOP (QK^T and PV only; softmax FLOPs are not counted). Single-thread variants ran with one thread; Windows is free to place it on any core, so it is not pinned. All variants pass `make check` on all four inputs (max error ≤ 7e-6 against a limit of 1e-4).

Q1. Machine

CPU: Intel Core Ultra 7 155H (Meteor Lake) in a Lenovo Yoga Slim 7 (14IMH9), 28 W base power. **Cores:** 16 physical / 22 logical threads = 6 performance cores (Redwood Cove, Hyper-Threading, 12 threads) + 8 efficiency cores (Crestmont) + 2 low-power efficiency cores (Crestmont, on the SoC tile). **Clocks (Intel specification):** P-core 1.4 GHz base / 4.8 GHz max turbo; E-core 0.9 / 3.8 GHz; LP-E-core 0.7 / 2.5 GHz. The sustained all-core clock under load was not measured directly; see the caveat below.

SIMD: AVX2 with FMA, 256-bit vectors = 8 FP32 lanes, 16 vector registers of 256 bits; no AVX-512. **FMA units:** 2 per P-core (2×256-bit); the E-cores, to my understanding, execute 256-bit operations as two 128-bit halves on 2 FMA-capable pipes (16 FP32 FLOP/cycle; not verified by a measurement). **Caches:** L1d 48 KB per P-core and 32 KB per E-core; L2 2 MB per P-core, 2 MB per cluster of 4 E-cores, and 2 MB for the 2 LP-E cores; L3 24 MB shared; 64-byte lines. **Memory:** 32 GB soldered LPDDR5X-7467, 128-bit total bus (2 × 64-bit channels).

Peaks and ridge points

One core (P-core, 4.8 GHz): 4.8 GHz × 2 FMA units × 8 lanes × 2 FLOP/FMA = **153.6 GFLOP/s**.
All cores, summing each core type at its max turbo: P: 6 × 4.8 × 32 = 921.6; E: 8 × 3.8 × 16 = 486.4; LP-E: 2 × 2.5 × 16 = 80.0; total **1488 GFLOP/s**.
Memory bandwidth: theoretical 7467 MT/s × 16 B = 119.5 GB/s. Measured with a STREAM-triad (`tools/microbench.c`): 25.7 GB/s with one thread and **62.2 GB/s** with 22 threads (52% of theoretical). I use the measured 62.2 GB/s for the roofline, because it is what a program can actually obtain.
Ridge point = peak / bandwidth: one core 153.6 / 62.2 = **2.5 FLOP/B**; all cores 1488 / 62.2 = **23.9 FLOP/B** (with the theoretical 119.5 GB/s: 1.3 and 12.5).

> **Caveat.** Both compute peaks are upper bounds at the specification’s turbo clocks. This is a 28 W thin-and-light laptop, so single-core and especially all-core sustained clocks are lower, and the real percentage of the attainable peak is higher than the percentages below. A short FMA-only loop in `microbench` measured 86.1 GFLOP/s on one core, which is below what `attn_best` reached (86 GFLOP/s at N=1024 here, and 91.5 GFLOP/s in an earlier run), so that loop evidently did not run at full boost; I treat it as a floor, not a peak.

Q2. Naive baseline

variant	N	d	time (ms)	GFLOP/s	% of 1-core peak	speedup vs naive
naive	1024	64	106.03	2.53	1.6%	1.0×
naive	2048	64	436.42	2.46	1.6%	1.0×
naive	4096	64	1716.41	2.50	1.6%	1.0×
naive	4096	128	4104.15	2.09	1.4%	1.0×

Scaling. FLOPs are $4N^2d$, so time should grow ×4 when N doubles and ×2 when d doubles. Measured: N 1024→2048 gives ×4.12, 2048→4096 gives ×3.93, both close to the ideal 4, so the naive code is quadratic in N and GFLOP/s is flat (2.5). Doubling d at N=4096 gives ×2.39 instead of 2: the extra 20% comes from step 3,

whose inner loop reads `V[j*d+k]` with a stride of d floats (256 B at d=64, 512 B at d=128), so every FMA touches a different cache line and the access pattern is worse when d is larger.

Where the time goes. The naive code reaches only 2.0–2.5 GFLOP/s, about 1.6% of one P-core's peak. Both matrix products accumulate into a single float. Without `-ffast-math` the compiler may not reorder floating-point additions, so it cannot vectorise the reduction, and every FMA depends on the previous one. An FMA has a latency of about 4 cycles, so a dependent chain executes 1 FMA per 4 cycles = 0.5 FLOP/cycle, i.e. about 2.4 GFLOP/s at 4.8 GHz — matching the measurement. In other words the time goes to FMA *latency* (one lane of 8 and one dependent chain, so about 1 of 2 FMA units is ever busy and only a fraction of the pipeline), not to throughput, and not mainly to memory. The remainder is the softmax: N^2 scalar `expf` calls plus three sweeps over the 64 MB matrix (a back-of-envelope estimate; I did not instrument the phases separately).

Q3. Optimisations

variant	N	d	time (ms)	GFLOP/s	% of 1-core peak	speedup vs naive
loops	1024	64	13.83	19.42	12.6%	7.7×
loops	2048	64	68.30	15.72	10.2%	6.4×
loops	4096	64	210.62	20.39	13.3%	8.1×
loops	4096	128	337.51	25.45	16.6%	12.2×
tiled	1024	64	7.49	35.83	23.3%	14.2×
tiled	2048	64	37.81	28.40	18.5%	11.5×
tiled	4096	64	151.90	28.28	18.4%	11.3×
tiled	4096	128	201.16	42.70	27.8%	20.4×
fused	1024	64	13.12	20.46	13.3%	8.1×
fused	2048	64	64.04	16.77	10.9%	6.8×
fused	4096	64	227.92	18.84	12.3%	7.5×
fused	4096	128	441.25	19.47	12.7%	9.3×
best	1024	64	3.12	86.18	56.1%	34.0×
best	2048	64	13.84	77.58	50.5%	31.5×
best	4096	64	56.71	75.73	49.3%	30.3×
best	4096	128	103.02	83.38	54.3%	39.8×

The "speedup" column is relative to naive for the same input. At N=4096, d=64: loops 8.1×, tiled 11.3×, fused 7.5×, best 30.3× (at d=128: 12.2×, 20.4×, 9.3×, 39.8×). `make check` output: 20 of 20 PASS (loops, tiled, fused, best, omp on 4 inputs).

(a) loops — 8.1×

What changed: K is transposed once, and the loops are reordered to *i,k,j* (scores) and *i,j,k* (output) so the innermost loop is `y[j] += a * x[j]`: unit stride, independent iterations, no reduction. The compiler vectorises it to 256-bit FMAs without needing to reassociate anything. **What limits it now:** each vector FMA needs a load of `s[j]`, a load of the other operand and a store of `s[j]` (2 loads + 1 store per FMA), so the loop is limited by load/store ports and L1/L2 bandwidth rather than by FMA throughput: 20 GFLOP/s = 13% of peak. The softmax is unchanged, so it still costs N^2 scalar `expf` and three passes over a 64 MB matrix.

(b) tiled — 11.3×

What changed: both products use one blocked GEMM (`gemm()`, written from scratch, no BLAS). A 6×16 tile of C lives in 12 vector registers across the whole k-loop; each loaded B vector feeds 6 FMAs and each broadcast A value 2, so there are about 0.7 loads per FMA instead of 3. A is packed in 96×128 blocks (48 KB, kept in L2) and B in 128×16 micro-panels (8 KB, kept in L1) and packed panels remove the 16 KB-stride cache-set conflicts of a 4096-float leading dimension. **What limits it now:** the GEMMs are fast but the surrounding work is not. The N×N matrix is still written once, passed over three times by the softmax, and read again (7 sweeps ≈ 470

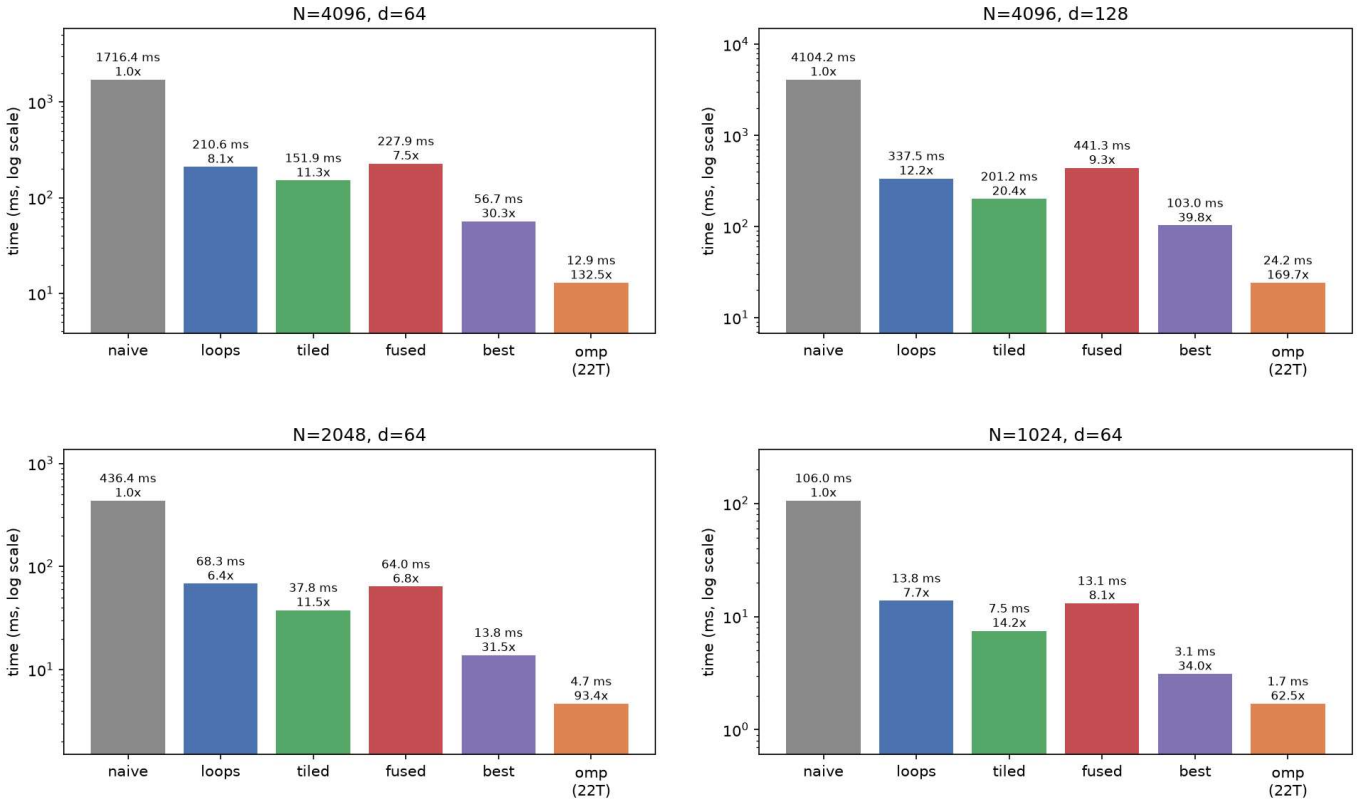
MB, about 18 ms at the measured 25.7 GB/s single-thread bandwidth, i.e. only ~12% of the 152 ms), and the softmax uses scalar `expf`. By subtraction (if the two GEMMs ran at about 80 GFLOP/s, as in `best`, they would take about 54 ms), roughly 100 ms — two thirds of the run — is softmax, about 6 ns (≈26 cycles) per score element. This is an inference, not a direct measurement.

(c) fused — 7.5×

What changed: for each query row, scores → softmax → output row, using one buffer of N floats (16 KB for N=4096, resident in L1). The N×N matrix is never stored, so DRAM traffic falls from about 470 MB to about 4 MB (Q, K, V, O only). **What limits it now:** there is no register reuse: one query row at a time means each K^T or V element fetched from L2 feeds a single FMA, so the inner loops are bound by load ports and L2 bandwidth (the same 2-loads-per-FMA pattern as `loops`), and `expf` is still scalar. It is therefore *slower* than `tiled` (18.8 vs 28.3 GFLOP/s at N=4096, d=64) although it moves far less memory: fusion fixes the DRAM traffic, tiling fixes on-core efficiency, and neither alone is enough.

(d) best — 30.3× (75.7 GFLOP/s, 49% of the 153.6 turbo peak)

What I did: combined fusion and tiling and removed the remaining scalar work. (1) Q, K and V are packed into panels; K and V once per call, Q per row block. (2) A block of 30 query rows is processed at a time at N=4096 (6 rows × 5 tiles, sized so the score block is ≤ 512 KB and stays in L2). Each packed K or V panel is therefore reused by 30 rows from cache instead of 1, which fixes the fused variant's weakness, while the N×N matrix still never exists. (3) The same 6×16 register-tile micro-kernel computes both products. (4) The softmax runs on the L2-resident block with a hand-vectorised `exp`: range reduction by round-to-nearest, a degree-6 polynomial, and 2^n built directly in the exponent bits, with relative error about 1e-7 (the check allows 1e-4); the max and sum passes are also vectorised. (5) The 1/rowsum is applied once to the d outputs instead of to the N probabilities, saving a pass and N/d times fewer multiplies. (6) Only GCC/Clang vector extensions are used (no intrinsics), so the file maps to AVX-512, AVX2 or NEON.
What limits it now: FMA issue rate and the softmax: the two matrix products need 4d = 256 FLOP per score element while the exp pass needs about 25 operations; at 49% of the turbo peak the kernel is already close to what a laptop core sustains at lower clocks. Remaining overheads are edge tiles, packing, and the max/exp/sum passes.



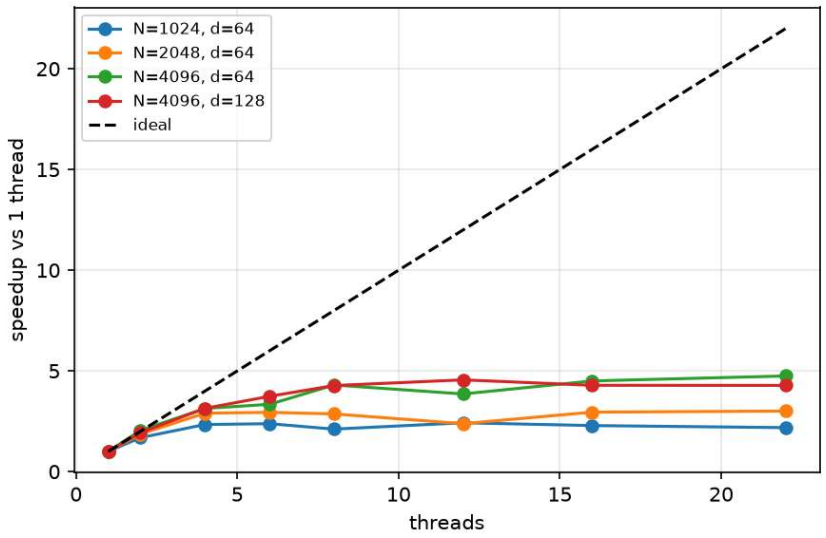
Q4. OpenMP

Implementation (`attn_omp.c`): `attn_best` with the 30-row blocks distributed by `#pragma omp for schedule(dynamic,1)` (dynamic because the P-, E- and LP-E-cores run at different speeds, so static splitting would leave fast cores idle). K and V are packed once, in parallel, and shared read-only; every thread has private Q-panel and score-block scratch, so there is no write sharing, no atomics and no false sharing. The block size shrinks for small N so that each thread gets at least 4 blocks. Thread count is set with `OMP_NUM_THREADS`.

threads	N=1024, d=64	N=2048, d=64	N=4096, d=64	N=4096, d=128
	ms / speedup / efficiency	ms / speedup / efficiency	ms / speedup / efficiency	ms / speedup / efficiency
1	3.71 / 1.00× / 100%	14.05 / 1.00× / 100%	61.51 / 1.00× / 100%	103.48 / 1.00× / 100%
2	2.20 / 1.69× / 84%	7.54 / 1.86× / 93%	30.07 / 2.05× / 102%	53.49 / 1.93× / 97%
4	1.58 / 2.34× / 58%	4.84 / 2.90× / 73%	19.62 / 3.14× / 78%	32.94 / 3.14× / 79%
6	1.56 / 2.38× / 40%	4.77 / 2.94× / 49%	18.40 / 3.34× / 56%	27.68 / 3.74× / 62%
8	1.76 / 2.11× / 26%	4.90 / 2.86× / 36%	14.30 / 4.30× / 54%	24.21 / 4.27× / 53%
12	1.53 / 2.42× / 20%	5.90 / 2.38× / 20%	15.95 / 3.86× / 32%	22.71 / 4.56× / 38%
16	1.62 / 2.29× / 14%	4.76 / 2.95× / 18%	13.66 / 4.50× / 28%	24.14 / 4.29× / 27%
22	1.70 / 2.18× / 10%	4.67 / 3.01× / 14%	12.95 / 4.75× / 22%	24.18 / 4.28× / 19%

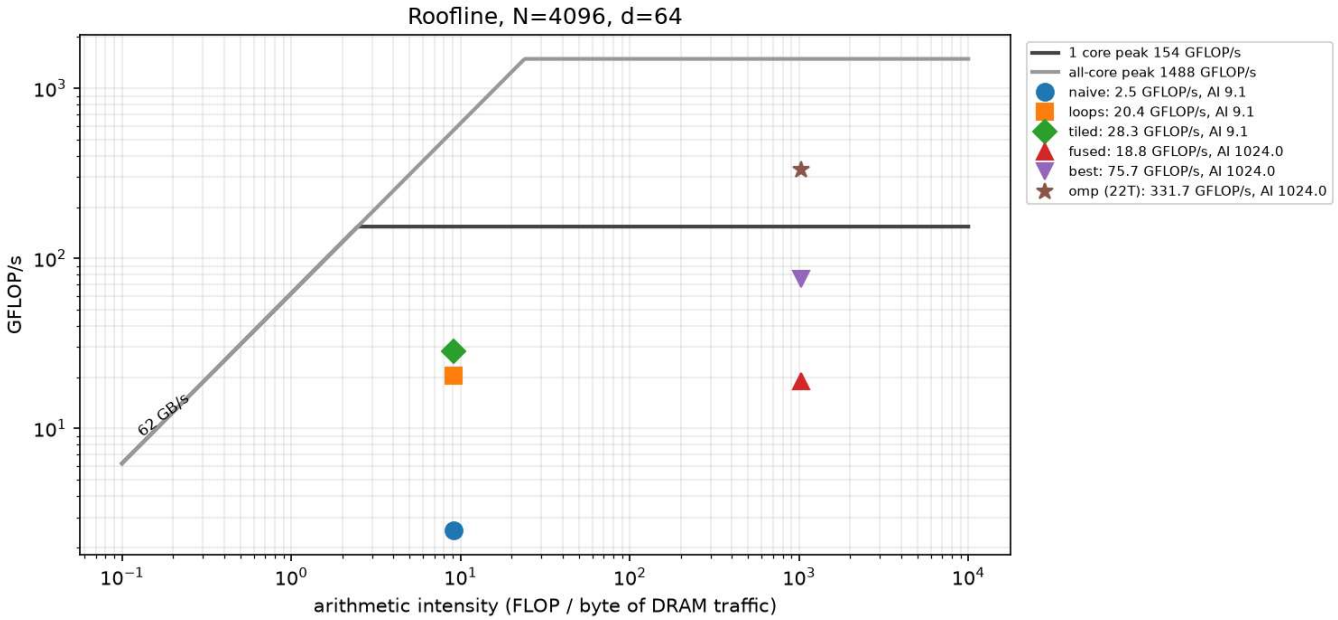
Efficiency = speedup / threads relative to the same program on one thread. For the absolute rate:

threads	N=1024, d=64	N=2048, d=64	N=4096, d=64	N=4096, d=128
1	72.5 (4.9% of all-core peak)	76.4 (5.1% of all-core peak)	69.8 (4.7% of all-core peak)	83.0 (5.6% of all-core peak)
2	122.2 (8.2% of all-core peak)	142.5 (9.6% of all-core peak)	142.8 (9.6% of all-core peak)	160.6 (10.8% of all-core peak)
4	169.5 (11.4% of all-core peak)	221.9 (14.9% of all-core peak)	218.9 (14.7% of all-core peak)	260.7 (17.5% of all-core peak)
6	172.4 (11.6% of all-core peak)	225.0 (15.1% of all-core peak)	233.4 (15.7% of all-core peak)	310.3 (20.9% of all-core peak)
8	152.9 (10.3% of all-core peak)	219.0 (14.7% of all-core peak)	300.3 (20.2% of all-core peak)	354.8 (23.8% of all-core peak)
12	175.7 (11.8% of all-core peak)	182.0 (12.2% of all-core peak)	269.2 (18.1% of all-core peak)	378.3 (25.4% of all-core peak)
16	165.7 (11.1% of all-core peak)	225.7 (15.2% of all-core peak)	314.4 (21.1% of all-core peak)	355.8 (23.9% of all-core peak)
22	158.2 (10.6% of all-core peak)	229.9 (15.4% of all-core peak)	331.7 (22.3% of all-core peak)	355.2 (23.9% of all-core peak)



Scaling. At N=4096, d=64, two threads give 2.05× (102%), four give 3.14× (78%), six 3.34× (56%), eight 4.30×, and 22 threads 4.75× (22% efficiency) — 331.7 GFLOP/s, i.e. 132× the naive code and 22% of the 1488 GFLOP/s all-core turbo ceiling. Interpretation (consistent with the data; I did not instrument core placement): (i) scaling is near-linear to 2 threads, then efficiency falls as more cores are active because the chip shares a 28 W budget and the all-core clock drops below single-core turbo; (ii) beyond about 6 threads additional threads are either hyper-threads sharing a P-core (the FMA units are already busy, so they add little) or E-/LP-E cores, which have half the FMA width (128-bit pipes) and lower clocks, so each adds much less than a P-core; (iii) the curve is therefore concave and nearly flat from 8 to 22 threads, with non-monotonic points (e.g. 12 threads < 8 or 16) that are within the scheduling and thermal noise of a laptop. Memory is not the limit: the fused kernel has an arithmetic intensity of about 1024 FLOP/B against an all-core ridge point of 24. Smaller inputs scale worse (N=1024 peaks at 175 GFLOP/s and flattens after 4 threads): there are only a few blocks per thread, and the serial parts (allocation, thread start-up, the parallel packing barrier) are a larger fraction of a 1.5 ms run.

Q5. Roofline



The plot uses the Q1 numbers (153.6 GFLOP/s one core, 1488 GFLOP/s all cores, 62.2 GB/s measured bandwidth); each variant is a point at N=4096, d=64 with y = measured GFLOP/s (the same $4N^2d = 4.295$ GFLOP for every variant) and x = its arithmetic intensity (AI) in FLOP per byte of DRAM traffic. The omp point is the 22-thread run.

How the arithmetic intensity was computed

AI = FLOP / bytes. The bytes are a **model** of compulsory DRAM traffic, not a hardware-counter measurement:

naive, loops, tiled (the $4N^2$ = 67 MB score matrix is far larger than the 24 MB L3, so every sweep goes to DRAM): the matrix is written by step 1 (1 sweep), read by the max pass (1), read and written by the exp pass (2), read and written by the normalise pass (2), and read by step 3 (1): 7 sweeps × 67.1 MB = 470 MB, plus Q, K, V read and O written (16Nd = 4.2 MB). AI = 4.295 GFLOP / 474 MB = **9.06 FLOP/B**.
fused, best, omp: only Q, K, V are read and O is written (K, V and the score block stay in cache): 16Nd = 4.2 MB, AI = N/4 = **1024 FLOP/B**.

What limits each variant

Ridge points: 2.5 (one core) and 23.9 FLOP/B (all cores). The unfused variants (AI 9.06) are above the one-core ridge, so a single thread is compute-side limited; but they sit *below* the all-core ridge, so a fully parallel unfused implementation would be bandwidth-limited at 9.06 × 62.2 = 563 GFLOP/s — another reason to fuse. Fused variants (AI 1024) are far to the right of both ridges: DRAM is irrelevant and only compute can limit them.

naive (2.5 GFLOP/s): about 1.6% of the one-core peak and far below every roof; limited by the latency of the dependent scalar FMA chain (Q2), not by memory. **loops** (20.4): vectorised but limited by load/store ports (2 loads + 1 store per FMA) and by the scalar softmax. **tiled** (28.3): the GEMMs are near the compute roof but the run is dominated by scalar `expf` and the seven sweeps over S; the 470 MB of traffic at 25.7 GB/s is only ~18 ms of 152 ms, so even here DRAM is a minor term (single-thread bandwidth roof at AI 9.06: 233 GFLOP/s, far above the measured point). **fused** (18.8): AI is high but the achieved rate is low: one row at a time gives no register reuse, so it is limited by L1/L2 load bandwidth and scalar `expf`. **best** (75.7): 49% of the one-core turbo peak, limited by FMA issue and the exp/max/sum passes. **omp, 22 threads** (331.7): 22% of the all-core turbo ceiling and about 2.2× the single P-core roof, limited by the mix of core types, the shared power/thermal budget and parallel overheads (Q4), not by memory. Moving right (fusion) removed the DRAM term; moving up (register tiling, vectorised exp, threads) approached the compute roofs.

Notes and limitations

(1) Peak numbers are specification-based upper bounds; the laptop's sustained clocks are lower, so the percentages of peak are conservative. (2) Arithmetic intensities are a traffic model (compulsory DRAM bytes), not counter measurements. (3) Statements about where the time goes in naive and tiled are derived from rates and subtraction, not from per-phase instrumentation. (4) Hybrid-core placement was left to the Windows scheduler, so run-to-run variation exists (e.g. a few isolated omp points deviate from the trend); I report medians of 5 invocations from a single quiet session.